PLAN DE ASEGURAMIENTO DE CALIDAD DE SOFTWARE (SQA)

Estándar IEEE 730-2014

Información del Documento	Detalle
Proyecto	Sistema de Soporte UMG
Documento	Plan de Aseguramiento de Calidad (SQA)
Estándar	IEEE Std 730-2014
Versión	1.0.0
Estado	Aprobado
Organización	Universidad Mariano Gálvez (UMG)

---

1. Propósito y Alcance

1.1 Propósito

El presente Plan de Aseguramiento de Calidad de Software (SQA) establece las actividades, procesos, responsabilidades y estándares requeridos para garantizar que la API RESTful del **Sistema de Soporte UMG** cumpla con los niveles de calidad, seguridad, mantenibilidad y rendimiento especificados para la gestión de incidentes y solicitudes ITIL.

1.2 Alcance

Este plan abarca todas las fases del Ciclo de Vida del Desarrollo de Software (SDLC) para la API del Sistema de Soporte UMG, incluyendo:

- Definición de requerimientos y especificación BDD (Gherkin).
- Diseño del contrato de API RESTful (OpenAPI 3.0.3).
- Desarrollo del backend en Node.js/Express y persistencia en Neon PostgreSQL.
- Pruebas automatizadas (Unitarias, Integración y End-to-End).
- Gestión de defectos y despliegue continuo.

---

2. Documentos de Referencia

- **IEEE Std 730-2014:** IEEE Standard for Software Quality Assurance Processes.
- **IEEE Std 1044-2009:** IEEE Standard for Classification for Software Anomalies.
- **ISO/IEC/IEEE 29119-3:2021:** Software and systems engineering — Software testing — Part 3: Test documentation.
- **RFC 9110:** HTTP Semantics.

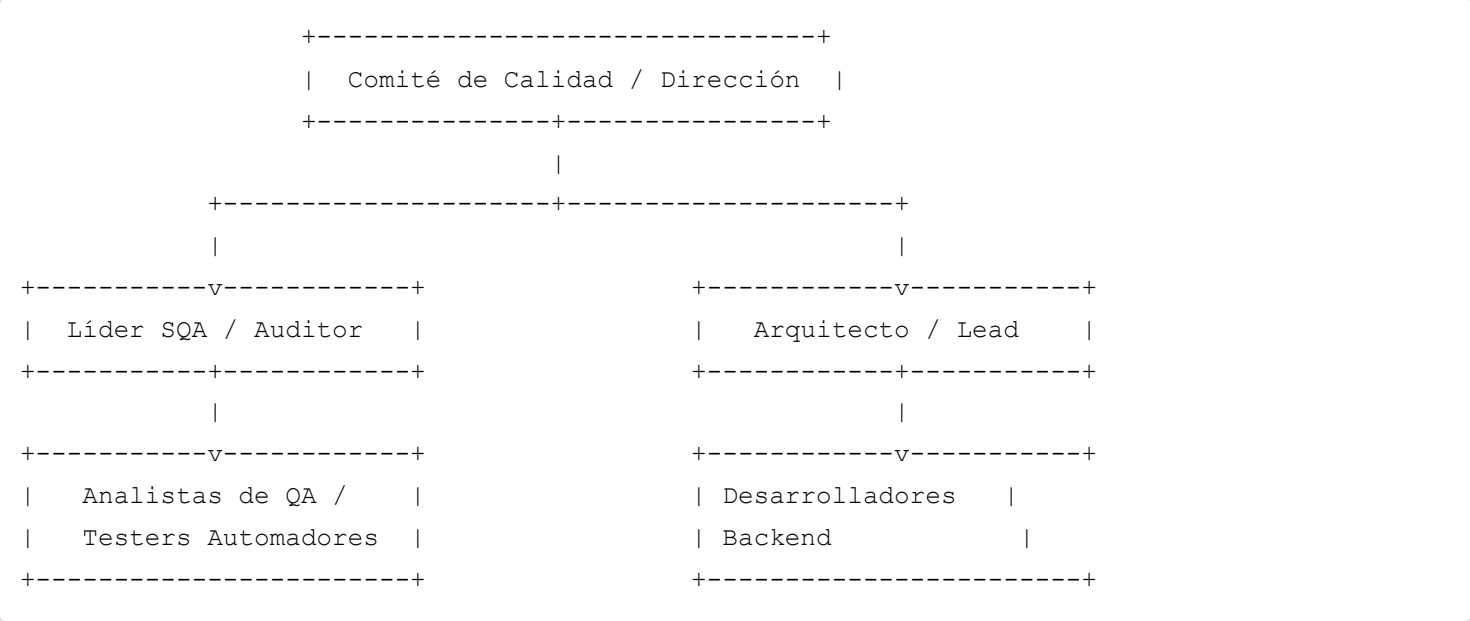
- **RFC 9457:** Problem Details for HTTP APIs.
- **OpenAPI Specification 3.0.3:** Swagger / Linux Foundation.

---

3. Gestión y Organización de SQA

3.1 Estructura Organizativa

Las actividades de SQA son independientes del equipo de construcción del código para evitar conflictos de interés y garantizar imparcialidad.



3.2 Roles y Responsabilidades

Rol	Responsabilidades SQA
Líder SQA	Diseñar, mantener y auditar el cumplimiento de este plan; reportar métricas de calidad a dirección.
Analista QA / Tester	Diseñar y ejecutar casos de prueba (ISO 29119-3), validar BDD y registrar defectos (IEEE 1044).
Arquitecto de Software	Garantizar el cumplimiento de patrones de arquitectura (3 capas), estándares REST y seguridad JWT.
Desarrollador Backend	Escribir código adherido a estándares, realizar pruebas unitarias ($\ge 80%$ cobertura) y corregir bugs.

---

4. Documentación del Software y Productos de Trabajo

El equipo de SQA auditará los siguientes entregables para asegurar su actualización e integridad:

1. **Especificación de Requerimientos y Criterios BDD:** Archivos `.feature` en sintaxis Gherkin.
2. **Contrato de API:** Archivo `openapi.yaml` bajo norma OpenAPI 3.0.3.
3. **Casos de Prueba:** Especificación formal bajo ISO/IEC/IEEE 29119-3:2021.
4. **Reportes de Cobertura de Código:** Reportes generados automáticamente por Jest.
5. **Bitácora de Defectos:** Registro estandarizado bajo IEEE 1044-2009.

5. Estándares, Prácticas y Métricas

5.1 Estándares de Codificación y Diseño

- **Formato de Código:** Configuración obligatoria de ESLint y Prettier.
- **Manejo de Errores:** Adopción de la especificación RFC 9457 (`application/problem+json`).
- **Seguridad:** Uso de Hash de contraseñas (Bcrypt) y autenticación stateless via JWT (Bearer Token).

5.2 Métricas de Calidad

Se evalúan las siguientes métricas en cada ciclo de integración:

- **Cobertura de Código (Code Coverage):** $$\text{Cobertura Total} = \left(\frac{\text{Líneas Ejecutadas en Tests}}{\text{Líneas Totales del Código}} \right) \times 100 \ge 80\%$$
- **Densidad de Defectos:** $$\text{Densidad} = \frac{\text{Número Total de Defectos}}{\text{KLOC (Miles de Líneas de Código)}}$$
- **Tasa de Cierre de Defectos (Defect Fix Rate):** $$\text{Fix Rate} = \left(\frac{\text{Defectos Resueltos}}{\text{Defectos Totales Reportados}} \right) \times 100 \ge 90\%$$

---

6. Revisiones y Auditorías de Software

Tipo de Review	Frecuencia	Responsable	Entregable
Code Review (PR)	Por cada Pull Request	Lead Dev / Peer Reviewer	Aprobación en GitHub/GitLab
Auditoría de Seguridad	Semanal	Líder SQA	Scan de vulnerabilidades (`npm audit`)
Auditoría de Arquitectura	Por Sprint	Arquitecto de Software	Checklist de arquitectura de 3 capas
Revisión SQA de Proceso	Quincenal	Líder SQA	Reporte de cumplimiento IEEE 730

---

7. Estrategia de Pruebas y V&V (Shift-Left & Pirámide)

Se aplica la estrategia **Shift-Left**, integrando validaciones desde la fase de requerimientos con lenguaje Gherkin. La ejecución de pruebas automatizadas sigue la estructura de la **Pirámide de Pruebas**:

1. **Pruebas Unitarias (Jest):** Reviso directo de funciones puras y lógica de negocio (SLA, fechas).
2. **Pruebas de Integración (Supertest + PostgreSQL):** Verificación de endpoints, rutas, middlewares y consultas/transacciones en Neon PostgreSQL.
3. **Pruebas End-to-End (Postman/Newman):** Validación de flujos de negocio completos multiparticipante.

---

8. Control de Herramientas y Entorno

Categoria	Herramienta Utilizada	Uso en el Proyecto
Control de Versiones	Git / GitHub	Control de código fuente y ramas (`main` , `develop` , `feature/*`).
Análisis Estático	ESLint / Prettier	Inspección de estilo de código y prevención de errores de sintaxis.
Testing Framework	Jest + Supertest	Suite automatizada de pruebas unitarias y de integración HTTP.
Base de Datos Test	Neon PostgreSQL (Branch de Test)	Base de datos aislada para pruebas de integración sin afectar producción.
Integración Continua	GitHub Actions	Ejecución automatizada de linter, tests y build en cada `push` / `PR` .

9. Gestión de Riesgos en SQA

Riesgo de Calidad	Probabilidad	Impacto	Plan de Mitigación
Caída o lentitud de la base de datos en la nube (Neon DB).	Media	Alto	Configuración de Pool de conexiones con reconexión automática y Mocks para pruebas unitarias.
Incompatibilidad entre el frontend y las respuestas de la API.	Media	Alto	Adopción estricta de la filosofía API-First y contrato OpenAPI 3.0.3 validado.
Fuga de secretos o tokens JWT en repositorios públicos.	Baja	Crítico	Uso de variables de entorno (`.env`), `.gitignore` estricto y escaneo con `git-secrets` .